INSTITUT NATIONAL
DES SCIENCES
APPLIQUÉES
ROUEN NORMANDIE

Département GM

PROJET ANALYSE NUMERIQUE

Verlet et Runge-Kutta pour une équation differentielle d'ordre 2

Encadrant :
M. J.G.CAPUTO

Présenté par :
Mouad SHERADJ
DRISSI
Amr ZAKI SALIH

Année universitaire : 2024-2025

Remerciements

Nous tenons à remercier Monsieur Caputo pour son accompagnement et pour l'opportunité qu'il nous a offerte à travers ce projet d'analyse numérique. Ce travail nous a permis de mettre en pratique les concepts théoriques étudiés en classe, tout en approfondissant notre compréhension de l'importance des méthodes numériques dans la résolution de problèmes complexes. Cette expérience a également renforcé nos compétences en programmation et en analyse mathématique, contribuant ainsi à notre développement académique et professionnel.

Table des matières

1 Introduction . . . 3
2 Méthode de Verlet . . . 3
2.1 Présentation et formulation mathématique . . . 3
2.2 Démonstration de la formule de Verlet . . . 4
2.3 Discrétisation et adaptation de la méthode au système . . . 4
2.4 Algorithme de la première approche . . . 6
2.5 Algorithme de la seconde approche . . . 6
3 Méthode de Runge Kutta 4 . . . 6
3.1 Présentation de la méthode RK4 . . . 6
3.2 Explication du code d'implémentation . . . 8
4 Explication du code principal . . . 8
4.1 Explication de la première partie . . . 8
4.2 Explication de la seconde partie . . . 11
5 Test et interpretation des résultats . . . 12
5.1 Partie 1 . . . 12
5.2 Partie 2 . . . 17
5.3 Interprétation des courbes des résultats numériques . . . 21
6 Conclusion . . . 22

1 Introduction

Ce rapport présente une étude approfondie des méthodes numériques pour la résolution d'équations différentielles d'ordre 2, en mettant l'accent sur les méthodes de Verlet et de Runge-Kutta d'ordre 4. Ces méthodes sont largement utilisées dans des domaines scientifiques et techniques pour modéliser des systèmes dynamiques. L'objectif de ce travail est de comparer ces deux approches dans le contexte de la résolution d'équations différentielles d'ordre 2.

Nous commencerons par une présentation des deux méthodes. Ensuite, nous décrirons les algorithmes implémentés en Fortran pour résoudre deux types d'équations différentielles. Les résultats numériques obtenus seront analysés et comparés à des solutions théoriques pour évaluer la précision des méthodes. Enfin, nous discuterons des performances relatives de chaque méthode en fonction des paramètres du système.

2 Méthode de Verlet

Une équation différentielle d'ordre 2 s'écrit sous la forme générale :

$$\frac{d^2X}{dt^2} = F(X(t)), \quad \text{avec } X \in \mathbb{R}^n \text{ et } F \text{ une fonction de } \mathbb{R}^n \text{ vers } \mathbb{R}^n. \tag{1.1}$$

Pour résoudre numériquement l'équation différentielle (1), on introduit la fonction V définie par :

$$V(t) = X'(t),$$

ce qui permet de réécrire l'équation (1) sous la forme d'un système d'équations différentielles :

$$(S) : \begin{cases} V = X', \\ V' = F(X(t)). \end{cases}$$

2.1 Présentation et formulation mathématique

La méthode de Verlet est une méthode numérique pour intégrer les équations différentielles d'ordre 2. Mathématiquement, elle s'exprime par :

$$X(t+h) = 2X(t) - X(t-h) + h^2 F\big(X(t)\big),$$

où :
— $X(t)$ est la position à l'instant t,
— h est le pas de temps,
— $F(X(t)) = \frac{d^2X}{dt^2}$ est l'accélération.
Cette formule permet de calculer directement la position future en fonction des positions actuelles et passées, sans passer par la vitesse.

2.2 Démonstration de la formule de Verlet

D'après le développement de Taylor, on a les approximations suivantes pour $X(t+h)$ et $X(t-h)$:

$$X(t+h) = X(t) + hX'(t) + \frac{h^2}{2}X''(t) + O(h^3),$$
$$X(t-h) = X(t) - hX'(t) + \frac{h^2}{2}X''(t) + O(h^3).$$

En soustrayant ces deux relations, on obtient :

$$X(t+h) - X(t-h) = 2hX'(t) + O(h^3),$$

ce qui permet d'approcher la dérivée $X'(t)$ par :

$$X'(t) = \frac{X(t+h) - X(t-h)}{2h} + O(h^2). \tag{1.2}$$

Ainsi, la méthode de Verlet permet une approximation du premier ordre avec un ordre d'erreur de $O(h^2)$.

En développant à nouveau les équations de Taylor pour obtenir une expression de $X''(t)$, on a :

$$X(t+h) + X(t-h) = 2X(t) + h^2X''(t) + O(h^4),$$

ce qui permet d'approcher $X''(t)$ par :

$$X''(t) = \frac{X(t+h) + X(t-h) - 2X(t)}{h^2} + O(h^2).$$

Sachant que $X''(t) = F(X(t))$, on obtient la méthode de Verlet :

$$X(t+h) = -X(t-h) + 2X(t) + h^2F(X(t)).$$

2.3 Discrétisation et adaptation de la méthode au système

Pour tout i appartenant à $\{0, ..., n\}$, on pose les approximations suivantes :

$$X(t_i) \approx X_i, \quad X(t_{i+1}) \approx X_{i+1}, \quad V(t_i) \approx V_i, \quad F(X(t_i)) \approx F_i.$$

Le pas est défini par :

$$h = \frac{t_{\max} - t_{\min}}{n}, \quad t_{i+1} - t_i = h, \quad t_i = t_{\min} + ih.$$

D'où l'on trouve, pour la méthode de Verlet, la formule suivante :

$$X_{i+1} = -X_{i-1} + 2X_i + h^2F_i.$$

D'autre part, il existe une version de Verlet qui est plus adaptée à des systèmes utilisant la dérivée $\frac{dX}{dt}$, comme celui proposé dans la deuxième partie du sujet :

$$\frac{d^2X}{dt^2} = -\sin(X) - e\frac{dX}{dt}.$$

D'après la méthode de Verlet et la formule (2), on a :

$$X(t+h) = -X(t-h) + 2X(t) + h^2 F(X(t)).$$

$$\frac{2X(t+h)}{2h} = \frac{X(t+h) - X(t-h)}{2h} + \frac{2X(t) + h^2 F(X(t))}{2h}.$$

$$X(t+h) = hX'(t) + X(t) + \frac{h^2}{2} F(X(t)).$$

On a donc :

$$X(t+h) = hV(t) + X(t) + \frac{h^2}{2} F(X(t)).$$

Pour un t_i, la formule devient :

$$X_{i+1} = hV_i + X_i + \frac{h^2}{2} F_i.$$

D'autre part, on a :

$$X(t+h) - X(t-h) = 2hV(t) + O(h^3),$$

d'où une approximation :

$$V(t) \approx \frac{X(t+h) - X(t-h)}{2h}. \tag{1.3}$$

Et d'après l'équation de Verlet, on a :

$$X(t+2h) = 2X(t+h) - X(t) + h^2 F(X(t+h)). \tag{1.4}$$

En insérant cette expression dans la définition de la vitesse :

$$V(t+h) = \frac{X(t+2h) - X(t)}{2h}, \tag{1.5}$$

on obtient :

$$V(t+h) = V(t) + \frac{h}{2} \left(F(X(t)) + F(X(t+h)) \right). \tag{1.6}$$

Enfin, on obtient les relations de récurrence suivantes, qui définissent une seconde approche de Verlet :

$$\begin{cases} X_{i+1} = hV_i + X_i + \frac{h^2}{2} F_i, \\ V_{i+1} = V_i + \frac{h}{2} \left(F_i + F_{i+1} \right). \end{cases}$$

2.4 Algorithme de la première approche

Ici, on a comme données $n_{\max}, h, X(t=0), DX(t=0)$, mais on a besoin de $X(t+h)$ pour la première approche. On utilise la formule suivante pour calculer une approximation de $X(t+h)$:

$$X(t+h) = h\frac{DX(t)}{dt} + X(t) + \frac{h^2}{2}F(X(t))$$

```
donnees : nmax , h
conditions initiales : X1 = X(t=0) , DX1 = DX(t=0)
X2 = X1 + DX1 * h + 0.5 * F(X1) * h**2

Pour it = 1, nmax
   X3 = 2 * X2 - X1 + h^2 F(X2)
   t = (it +2 )h
   afficher(t, X3)
   X1 = X2
   X2 = X3
fin
```

2.5 Algorithme de la seconde approche

On dispose des données $n_{\max}, h, X(t=0), DX(t=0)$, mais nous avons besoin de $DX(t+h)$. Pour cela, on fait une approximation de la dérivée (vitesse) de X_2 en utilisant celle de X_1 (DX_1) .

```
donnees : nmax , h
conditions initiales : X1 = X(t=0) , DX1 = DX(t=0)
X2 = X1 + DX1 * h + 0.5 * F(X1,DX1) * h**2
DX2 = DX1 + 0.5 * h * [F(X1,DX1)+F(X2,DX1)]

Pour it = 1, nmax
   X3 = X2 + h * DX2 + 0.5 * h^2 * F(X2)
   DX3 = DX2 + 0.5 * h * ( F(X2,DX2) + F(X3,DX3) )
   afficher(t, X3, DX3)
   t = (it+2)h
   DX2=DX3
   X2 = X3
fin
```

3 Méthode de Runge Kutta 4

3.1 Présentation de la méthode RK4

La méthode de Runge Kutta est une méthode de résolution des équations différentielles à pas simple comme la méthode d'Euler. A l'inverse de la méthode d'Euler, celle de Runge Kutta permet une approximation beaucoup plus précise. Elle se caractérise par la discrétisation de l'intervalle h choisie c'est à dire, un découpage de cet intervalle en un nombre de parties égal à l'ordre de la méthode. L'ordre permet d'accroître le degré de précision.

On peut ainsi donner l'expression générale des méthodes de Runge Kutta :

$$k_i = f(t_n + c_i h; x_n + h \sum_{j=1}^{p} a_{ij} k_j)$$

$$x_{n+1} = x_n + \sum_{i=1}^{p} b_i k_i$$

avec $i = 1...p$ et p l'ordre de la méthode utilisée.

Ainsi la méthode de Runge Kutta est une étape dans le raffinement du calcul de l'intégrale de l'équation différentielle :

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_n+h} f(t, y(t))$$

avec $y(t)$ la fonction que l'on recherche et f une fonction connue suffisamment régulière pour que l'existence et l'unicité ne pose pas de problème.

En appliquant la méthode de Simpson, qui est une méthode consistant à remplacer la fonction intégrée par une parabole passant par les points extrêmes et le point milieu tel que :

$$\int_a^b f(x)dx \simeq \frac{b-a}{2}[f(a) + 4f(\frac{a+b}{2}) + f(b)]$$

nous obtenons ainsi :

$$\int_{t_n}^{t_{n+1}} f(t, y(t))dt \simeq \frac{h}{6}[f(t_n, y(t_n)) + 4f(t_{n+1/2}, y(t_{n+1/2})) + f(t_{n+1}, y(t_{n+1}))]$$

d'où la relation :

$$y_{n+1} = y_n + \frac{h}{6}[f(t_n, y_n) + 4f(t_{n+1/2}, y_{n+1/2}) + f(t_{n+1}, y_{n+1})]$$

Puis en utilisant Euler implicite et explicite, on peut minimiser l'erreur sur le calcul de $4f(t_{n+1/2}, y_{n+1/2})$ et on peut ainsi écrire que : $4f(t_{n+1/2}, y_{n+1/2} \simeq 2k_2 + 2k_3)$ avec :

- $k_1 = f(t_n, y_n)$
- $k_2 = f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1)$
- $k_3 = f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_2$

Et pour le terme $f(t_{n+1}, y_{n+1})$ nous l'approchons en estimant y_{n+1} par la méthode du point milieu, c'est-à-dire en appliquant la méthode du rectangle au milieu et on obteint finalement Runge-Kutta d'ordre 4 sous la forme :

$$y_{n+1} = y_n + h[\frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6}]$$

avec :

- $k_1 = f(t_n, y_n)$
- $k_2 = f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_1)$
- $k_3 = f(t_n + \frac{1}{2}h, y_n + \frac{1}{2}hk_2$
- $k_4 = f(t_n + h, y_n + hk_3)$

3.2 Explication du code d'implémentation

Dans le code ci dessous, on implémente la méthode de Runge-Kutta qui dans le main sera itéré. Ainsi dans un premier temps, il est nécessaire de calculer les différents k de la méthode, et pour ce faire, on appelle derivs, la subroutine qui définit les équations différentielle, pour calculer les coefficients k_1,k_2,k_3 ainsi que k_4. On a pour paramètre le temps t, n la dimension du système qui est égale à 2 ici, $x(n)$ le vecteur de l'état actuel, h le pas, les coefficients intermédiaires de la méthode de Runge-Kutta 4 k et finalement e qui sera utilisé dans la deuxième partie où l'on essaye de résoudre l'équation différentielle avec $\frac{d^2X}{dt^2} = -sin(X) - e\frac{dX}{dt}$ où $0 \leq e \leq 0.1$. Par ailleurs, on met à jour le temps ainsi que X qui est la combinaison pondérée classique de la méthode de Runge-Kutta.

```
1     subroutine rk4 (t,n,x,h,derivs,k1,k2,k3,k4,e)
2       implicit none
3       double precision x(n),k1(n),k2(n),k3(n),k4(n)
4       double precision h,t,e
5       integer n
6
7       call derivs(t,n,x,k1,e)
8       call derivs(t+0.5d0*h,n,x+0.5d0*h*k1,k2,e)
9       call derivs(t+0.5d0*h,n,x+0.5d0*h*k2,k3,e)
10      t = t+h
11      call derivs(t,n,x+h*k3,k4,e)
12
13      x = x + (h/6.d0)*(k1 + 2.d0*k2 + 2.d0*k3 + k4)
14
15      return
16    end
```

4 Explication du code principal

4.1 Explication de la première partie

Dans cette première partie, nous nous proposons de résoudre l'équation du pendule simple, qui est une équation différentielle du second ordre :

$$\frac{d^2X}{dt^2} = -\sin(X)$$

où :

— X est l'angle du pendule par rapport à la verticale ;
— $\frac{d^2X}{dt^2}$ est l'accélération angulaire.

Afin de résoudre cette équation, nous la reformulons en un système de deux équations différentielles du premier ordre en introduisant la vitesse angulaire, définie par :

$$\omega = \frac{dX}{dt}.$$

Ainsi, le système devient :
— $\frac{dX}{dt} = \omega$;
— $\frac{d\omega}{dt} = -\sin(X)$.

Dans notre code, nous représentons l'état du système par le vecteur X pour la méthode de Verlet et $X1$ pour la méthode RK4, chacun étant de taille 2, où :

— $X(1)$ correspond à X, la position angulaire ;
— $X(2)$ correspond à ω, la vitesse angulaire.

Le code principal est donné ci-dessous :

```
1   program main
2     implicit none
3     integer, parameter :: n=2
4     double precision X1(n),X2(n),X3(n),k1(n),k2(n),k3(n),k4(n)
5     double precision X(n)
6     double precision f1,f2,X0,DX0
7     double precision h,t1,t2,t
8     integer ntot,ih,it
9     external derivs
10
11    ! Lecture des valeurs initiales
12    read(*,*) t1
13    f1=cos(t1)
14    f2=-sin(t1)
15
16    ! Lecture des param tres
17    read(*,*) h
18    read(*,*) X0
19    read(*,*) DX0
20
21    do ih=1,6
22
23        h=h/10.d0
24        ntot=t1/h
25
26        X1(1)=X0
27        X1(2)=DX0
28        X(1)=X0
29        X(2)=DX0
30        X2(1) = X1(1) + X1(2) * h + 0.5d0 * (-sin(X1(1))) * h**2
31        X2(2) = X1(2) + 0.5d0 * h * (-sin(X1(1))-sin(X2(1)))
32
33        t2=2*h
34
35        do it=1,ntot
```

```
36              t=(it-1)*h
37              call rk4(t,n,X,h,derivs,k1,k2,k3,k4)
38              t2=(it+2)*h
39              call verlet(t2,n,h,X2,X1,X3,derivs)
40
41              X1=X2
42              X2=X3
43
44              if (ih==1) then
45                  write(10,*) t,X
46                  write(11,*) t2,X3
47              endif
48           enddo
49
50          write(*,*) "Pour verlet", h, abs(X3(1)-f1), abs(X3(2)-f2)
51          write(*,*) "Pour RK4", h, abs(X(1)-f1), abs(X(2)-f2)
52       enddo
53
54   end
```

Ensuite, dans la fonction `derivs`, nous calculons les dérivées de ces variables de la manière suivante :

- $dX(1)$ représente la dérivée de la position angulaire X, qui est égale à la vitesse angulaire ω ;
- $dX(2)$ représente la dérivée de la vitesse angulaire ω, qui est égale à l'accélération angulaire $-\sin(X)$.

Ces deux valeurs correspondent respectivement à la vitesse et à l'accélération du système.

Dans la subroutine Verlet, nous avons besoin de déterminer un $X2 = X(t = h)$ à l'initialisation. Pour cela, on utilise un développement limité d'ordre 2 de $X(t)$ autour de t_1 :

Pour une fonction $X(t)$ son développement limité d'ordre $X(t)$ autour de t_1 :

$$X(t_1 + h) = X(t_1) + hX'(t_1) + \frac{h^2}{2}X''(t1) + O(h^3)$$

En appliquant cette notation à X_1, nous obtenons que $X_1(1)$ représente $X(t_1)$ et $X_1(2)$ correspond à $X'(t_1)$. Ainsi, nous avons :

$$X_2(1) = X_1(1) + hX_1(2) + \frac{h^2}{2}(-sin(X_1(1)))$$

Dans le code, nous avons un pas de temps h qui contrôle la précision de la méthode, et les itérations permettent de faire progresser la solution pas à pas dans le temps pour résoudre l'équation différentielle.

Par ailleurs, nous initialisons des fonctions f_1 et f_2 correspondant aux solutions théoriques qui doivent être obtenues. Nous les comparons aux solutions calculées à l'aide des méthodes de Runge-Kutta et de Verlet. Plus cette différence est petite, plus la méthode est précise.

4.2 Explication de la seconde partie

La partie 2 est similaire à la partie 1, avec quelques modifications notables. Nous utilisons ici la deuxième approche de Verlet, qui est plus adaptée à l'équation différentielle proposée :

$$\frac{d^2X}{dt^2} = -\sin(X) - e\frac{dX}{dt}.$$

```
      program main
        implicit none
        integer,parameter :: n=2
        double precision X1(n),X2(n),X3(n),k1(n),k2(n),k3(n),k4(n)
        double precision X(n)
        double precision f1,f2,e,X0,DX0
        double precision h,t1,t2,t,omega,k
        integer ntot,ih,it
        external derivs

        ! Lecture des param tres initiaux
        read(*,*) t1
        read(*,*) e
        omega = sqrt(1.d0 - 0.25d0 * e**2)
        f1 = exp(-0.5d0 * e * t1) * cos(sqrt(1 - 0.25d0 * e**2) * t1)
        k = sqrt(1 - 0.25d0 * e**2) * sin(sqrt(1 - 0.25d0 * e**2) * t1)
        f2= -exp(-0.5d0 * e * t1) * (0.5d0 * e * cos(sqrt(1 - 0.25d0 * e**2)*t1) + k)

        ! Lecture des valeurs initiales
        read(*,*) h
        read(*,*) X0
        read(*,*) DX0

        do ih=1,6
          h = h / 10.d0
          ntot = t1 / h

          X1(1) = X0
          X1(2) = DX0

          X(1) = X0
          X(2) = DX0

          X2(1) = X1(1) + X1(2) * h + 0.5d0 * (-sin(X1(1)) - e * X1(2))
          X2(2) = X1(2) + 0.5d0 * h * (-sin(X1(1))-e * X1(2) - sin(X2(1))-e * X1(2))

          t2 = 2 * h

          do it = 1, ntot
            t = (it - 1) * h
            call rk4(t, n, X, h, derivs, k1, k2, k3, k4, e)
            t2 = (it + 2) * h
            call verlet(t2, n, h, X2, X3, derivs, e)

            X2 = X3

            if (ih == 1) then
```

```
48              write(10,*) t, X
49              write(11,*) t2, X3
50            endif
51
52          enddo
53
54          write(*,*) "Pour verlet", h, abs(X3(1) - f1), abs(X3(2) - f2)
55          write(*,*) "Pour RK4", h, abs(X(1) - f1), abs(X(2) - f2)
56        enddo
57      end
```

Par ailleurs, les fonctions f_1 et f_2 ont été adaptées pour prendre en compte ces modifications. En effet, on sait que si $e = 0$ alors la solution de l'équation différentielle est $X(t) = cos(t)$ et si $e \neq 0$ alors l'équation est celle d'un oscilatteur amorti non linéaire. Ainsi, pour de petites oscillations, on peut linéariser en approximant $sin(X) \simeq X$ et on obtient ainsi :

$$\frac{d^2X}{dt^2} + e\frac{dX}{dt} + X = 0$$

et c'est une équation différentielle classique dont la solution dépend de :

$$X(t) = e^{\frac{-et}{2}}(Acos(\omega t) + Bsin(\omega t))$$

avec

$$\omega = \sqrt{1 - \frac{e^2}{4}}$$

C'est ainsi pour ça que l'on a ces représentations de $f1$ et de $f2$ la dérivée de $f1$ pour pouvoir voir si cela concorde avec les résultats censés être obtenus.

5 Test et interpretation des résultats

Pour analyser les resultats obtenus lors des tests, nous avons code deux codes python qui comparent les méthodes numériques de Verlet et de Runge-Kutta d'ordre 4. Le premier code représente graphiquement les trajectoires obtenues par ces deux méthodes en fonction du temps. Le second code analyse l'erreur de chaque méthode en la comparant à la solution théorique. Si les instants de calcul diffèrent entre Verlet et RK4, une interpolation linéaire est appliquée pour aligner les données. Les résultats sont ensuite visualisés sous forme de courbes d'erreur, permettant d'évaluer la précision de chaque approche.

5.1 Partie 1

5.1.1 Test 1

Paramètres de test :
— $t_{\max} = 100.0$

— $h = 0.1$
— $X_0 = 1.0$
— $DX_0 = 0.0$

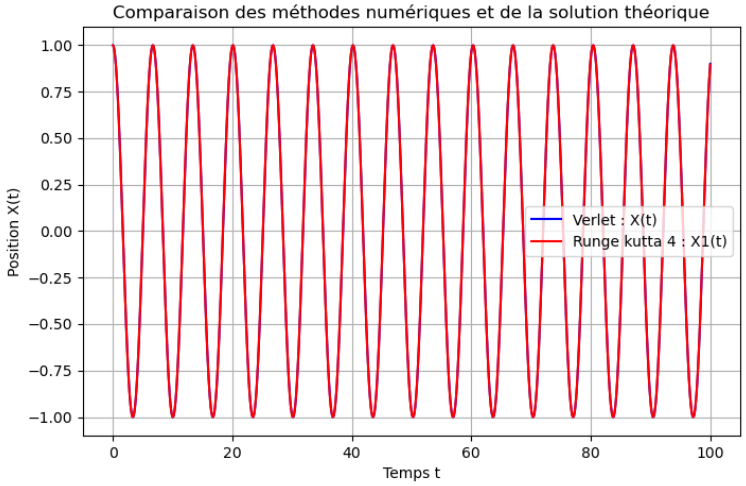


FIGURE 1.1 – Courbe 1 pour le Test 1

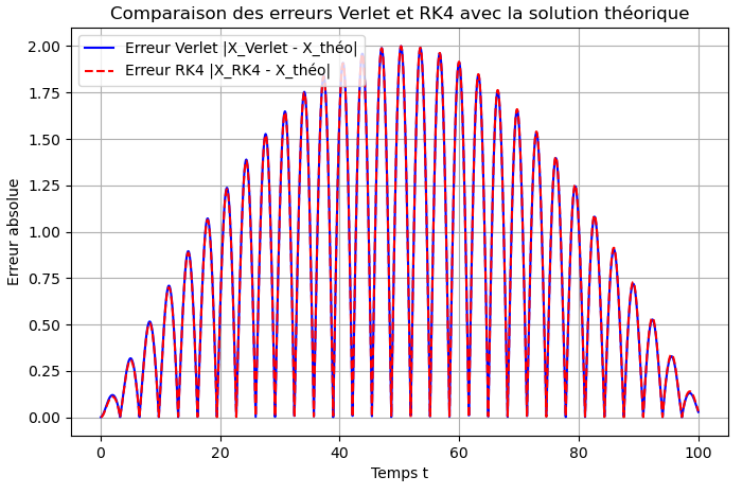


FIGURE 1.2 – Courbe différence pour le Test 1

5.1.2 Test 2

Paramètres de test :
— $t_{\max} = 100.0$
— $h = 5.0$
— $X_0 = 0.1$
— $DX_0 = 0.0$

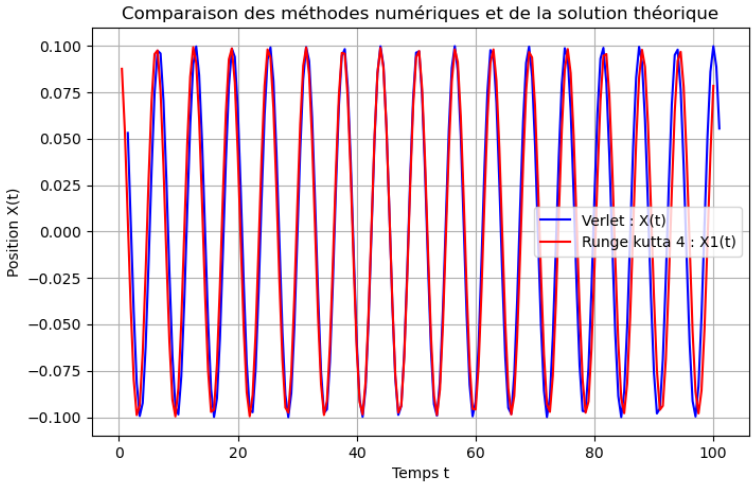


FIGURE 1.3 – Courbe 2 pour le Test 2

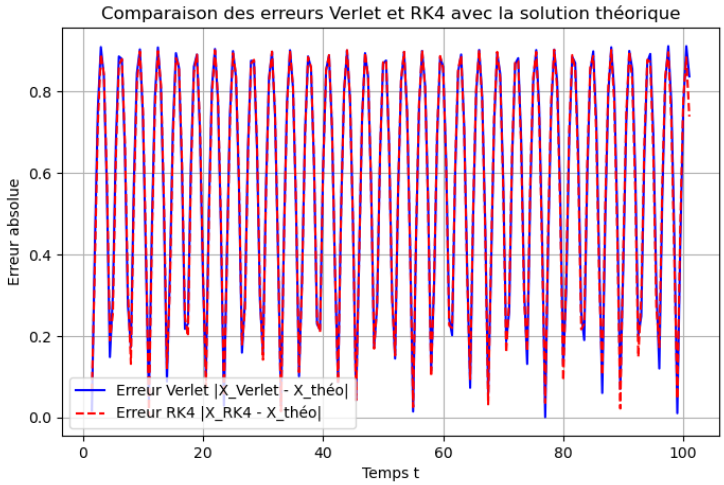


FIGURE 1.4 – Courbe différence pour le Test 2

5.1.3 Test 3

Paramètres de test :

— $t_{\max} = 100.0$
— $h = 5.0$
— $X_0 = 1.0$
— $DX_0 = 0.0$

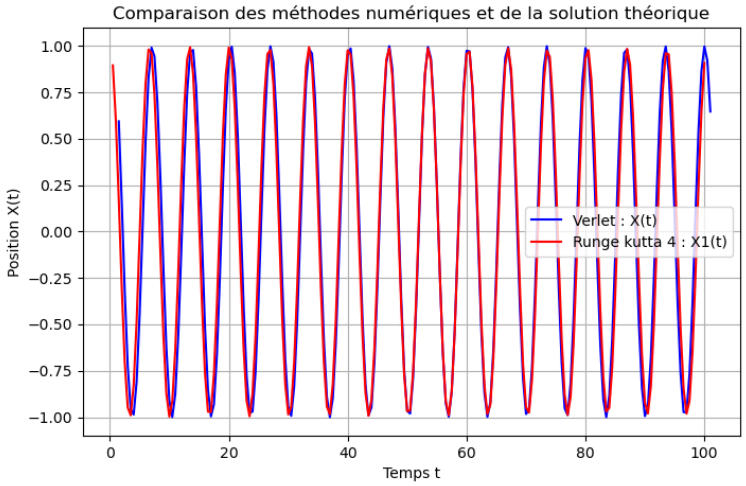


FIGURE 1.5 – Courbe 3 pour le Test 3

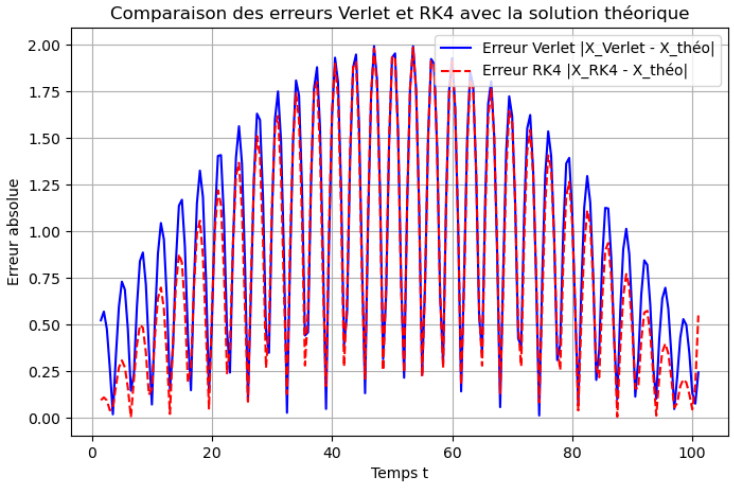


FIGURE 1.6 – Courbe différence pour le Test 3

5.1.4 Test 4

Paramètres de test :

— $t_{\max} = 100.0$
— $h = 2.0$
— $X_0 = 3.141592$
— $DX_0 = 0.0$

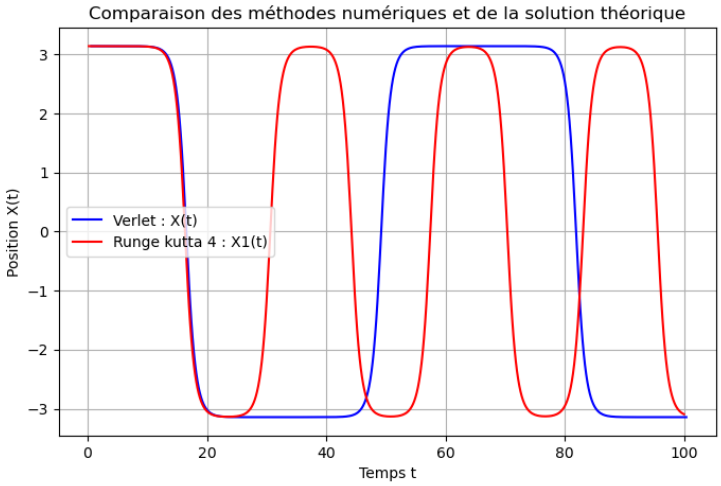


FIGURE 1.7 – Courbe 4 pour le Test 4

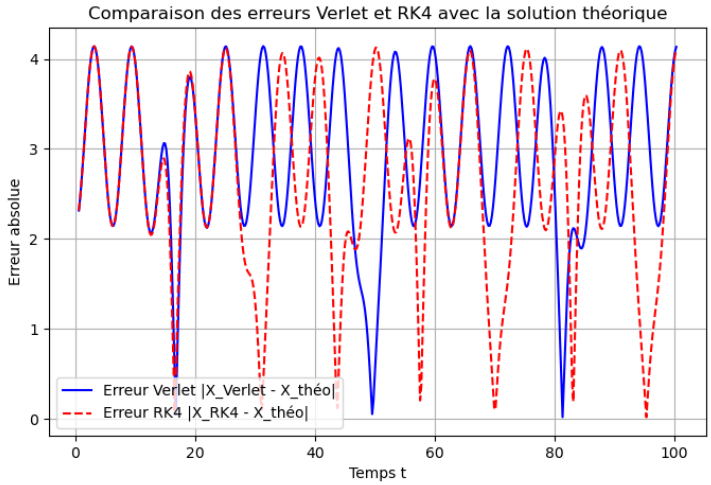


FIGURE 1.8 – Courbe différence pour le Test 4

5.1.5 Test 5

Paramètres de test :

— $t_{\max} = 100.0$
— $h = 0.1$
— $X_0 = -1.0$
— $DX_0 = 0.0$

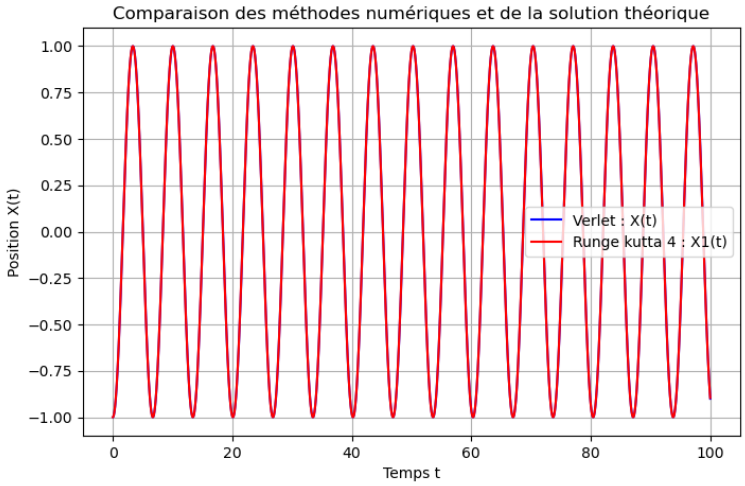


FIGURE 1.9 – Courbe 5 pour le Test 5

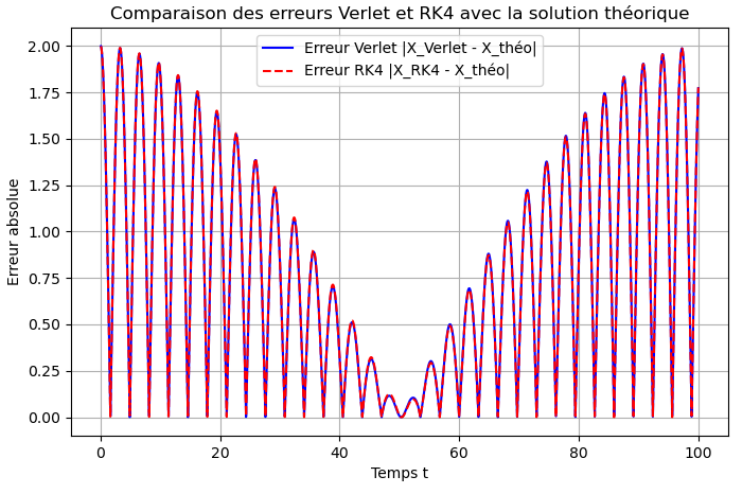


FIGURE 1.10 – Courbe différence pour le Test 5

5.2 Partie 2

5.2.1 Test 1

Paramètres de test :

- $t_{\max} = 100.0$
- $e = 0.05$
- $h = 0.1$
- $X_0 = 0.1$
- $DX_0 = 0.0$

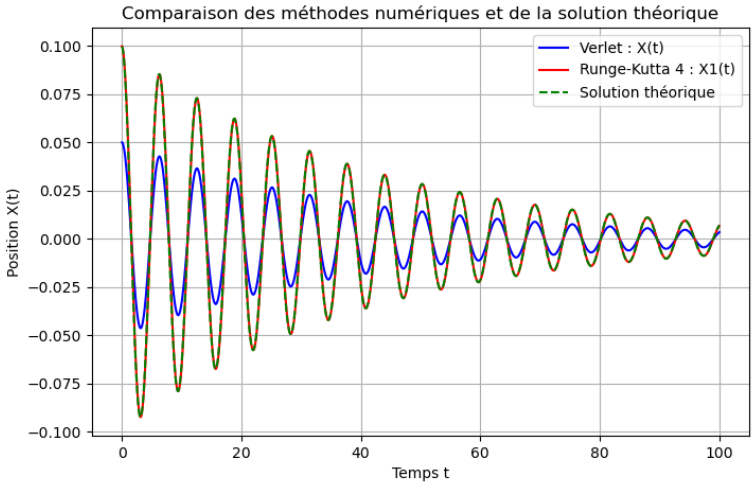


FIGURE 1.11 – Courbe 1 pour le Test 1 (Partie 2)

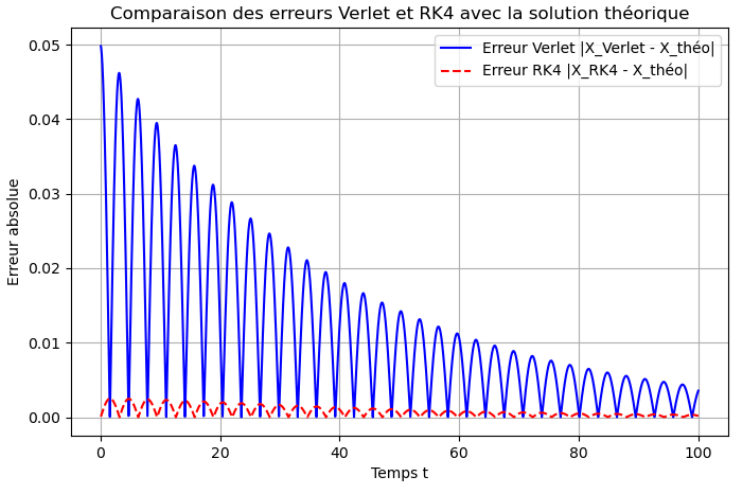


FIGURE 1.12 – Courbe différence pour le Test 1 (Partie 2)

5.2.2 Test 2

Paramètres de test :

— $t_{\max} = 100.0$
— $e = 0.05$
— $h = 0.1$
— $X_0 = 1.0$
— $DX_0 = 0.0$

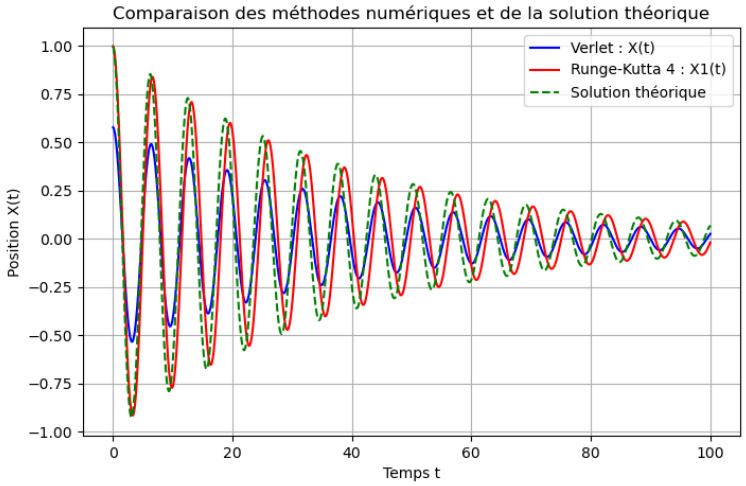


FIGURE 1.13 – Courbe 2 pour le Test 2 (Partie 2)

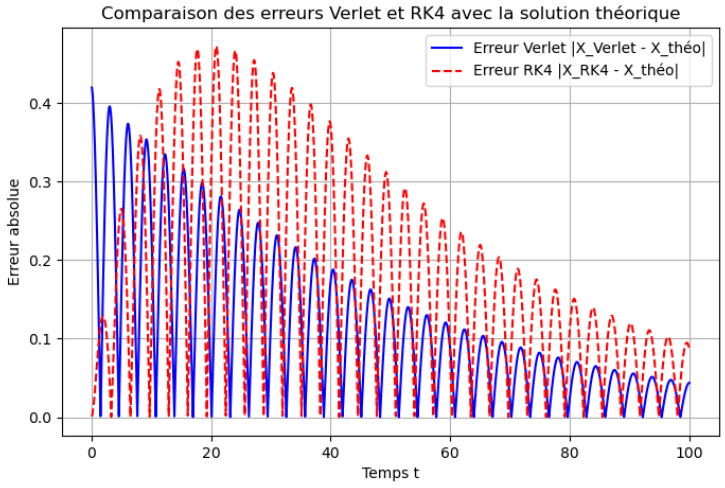


FIGURE 1.14 – Courbe différence pour le Test 2 (Partie 2)

5.2.3 Test 3

Paramètres de test :

— $t_{\max} = 100.0$
— $e = 0.01$
— $h = 1.0$
— $X_0 = 1.0$
— $DX_0 = 0.0$

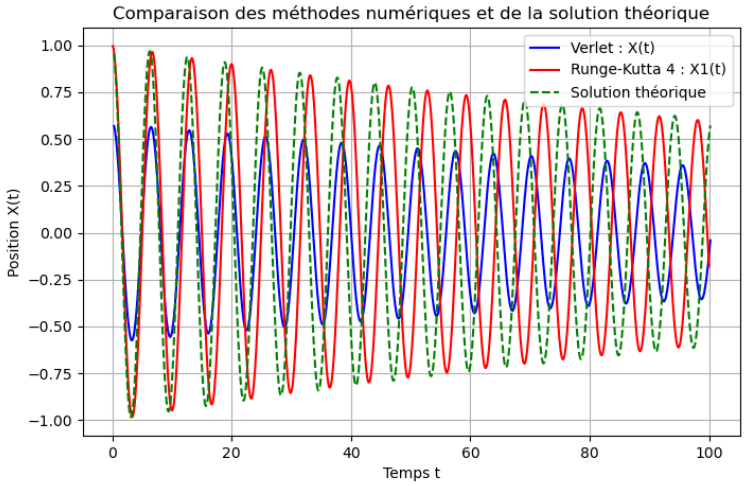


FIGURE 1.15 – Image Test pour le Test 3 (Partie 2)

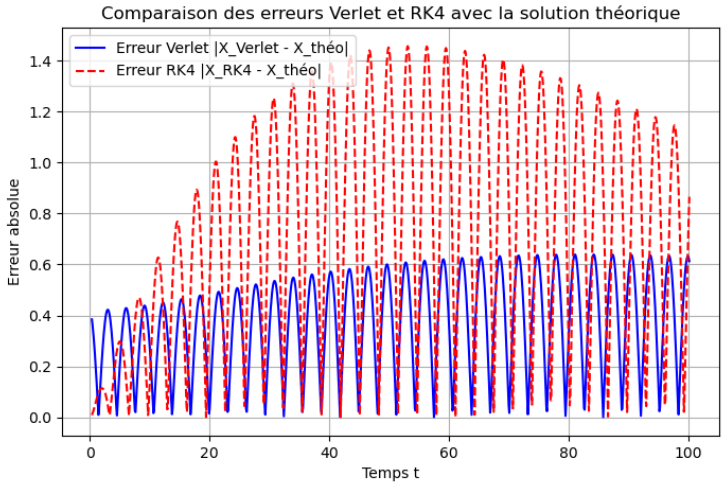


FIGURE 1.16 – Image différence pour le Test 3 (Partie 2)

5.2.4 Test 4

Paramètres de test :

— $t_{\max} = 100.0$
— $e = 0.05$
— $h = 0.1$
— $X_0 = 3.141592$
— $DX_0 = 0.0$

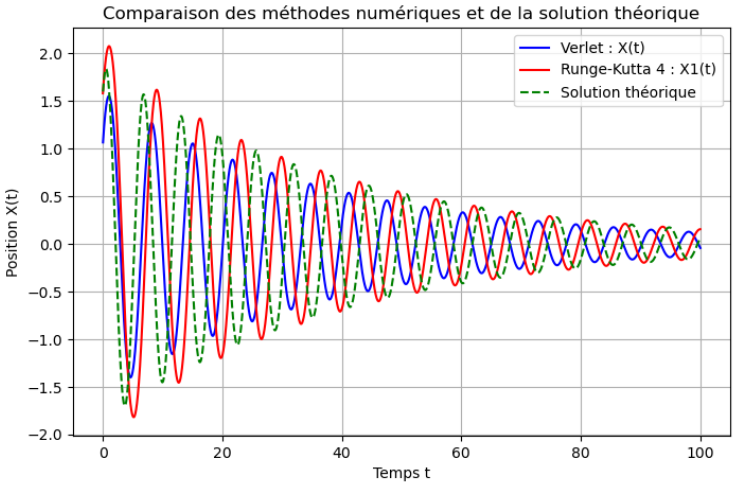


FIGURE 1.17 – Courbe 4 pour le Test 4 (Partie 2)

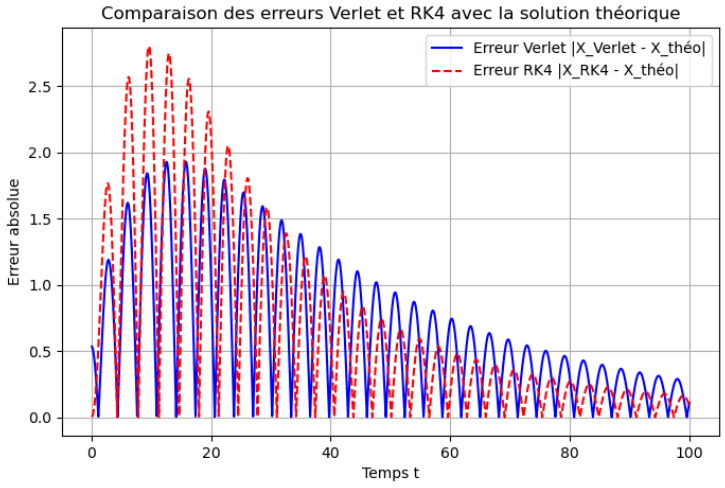


FIGURE 1.18 – Courbe différence pour le Test 4 (Partie 2)

5.3 Interprétation des courbes des résultats numériques

En ce qui concerne la première partie, les courbes obtenues avec les méthodes de Verlet et de Runge-Kutta d'ordre 4 montrent une dépendance marquée au pas de temps h. Pour des pas relativement petits ($h \leq 0.1$), les deux méthodes produisent des trajectoires oscillatoires proches de la solution exacte, avec une erreur comprise entre 0 et 2. Cependant, lorsque le pas de temps augmente ($h = 5.0$), la précision de la méthode de Verlet diminue, entraînant des déviations notables par rapport à Runge-Kutta 4, qui demeure plus stable malgré une légère perte de précision. De plus, pour certaines conditions initiales spécifiques, notamment $X_0 = \pi$, les erreurs numériques s'accumulent plus rapidement, soulignant la sensibilité des méthodes aux instabilités du système.

Dans la seconde partie, on observe que la méthode de Runge-Kutta 4 est globalement plus fiable. Cette différence s'explique par le fait que la méthode de Verlet approxime la vitesse en fonction de la position précédente, ce qui génère une erreur plus conséquente par rapport à Runge-Kutta 4. Dans ces cas, l'erreur accumulée par Verlet est plus importante, ce qui se traduit par une moins bonne prise en compte de la dissipation d'énergie et une divergence progressive avec la solution exacte. À l'inverse, la méthode de Runge-Kutta 4, en évaluant plusieurs étapes intermédiaires dans un même pas de temps, capture mieux la dynamique du système, y compris la décroissance des oscillations due à l'amortissement.

En conclusion, la méthode de Runge-Kutta 4 offre une meilleure précision globale, tandis que Verlet reste plus adaptée aux petits pas de temps et garantit une meilleure conservation de l'énergie du système sur le long terme.

6 Conclusion

Dans ce rapport, nous avons étudié et comparé les méthodes de Verlet et de Runge-Kutta d'ordre 4 en les appliquant à deux équations différentielles :

— $-sin(X)$, qui décrit un oscillateur harmonique conservatif
— $-sin(X) - e\frac{dX}{dt}$, qui introduit un amortissement contrôlé

Ces derniers tests ont permis de mettre en évidence que la méthode RK4 offrait une meilleure précision globale pour une large gamme de problèmes. En effet, dans l'équation de la partie 2, RK4 a mieux approximé la décroissance de l'amplitude due à l'amortissement, alors que Verlet, conçu principalement pour des systèmes conservatifs, a introduit des erreurs plus importantes dans la modélisation de cette dissipation.

Quant au système de la partie 1, la méthode de Verlet a été bien plus précise que celle de RK4 qui introduit une accumulation d'erreurs numériques. Cela confirme que la méthode de Verlet est particulièrement adaptée aux systèmes invariants.

Ainsi, pour conclure, si l'on recherche une solution précise sur un court intervalle de temps, en particulier en présence de termes dissipatifs, RK4 est souvent un meilleur choix que Verlet et au contraire, pour des simulations à longue durée où la conservation des invariants est cruciale, la méthode de Verlet s'avère plu fiable, malgré une précision locale qui peut parfois être inférieure due à son ordre.

Bibliographie

[1] Méthodes de Runge-Kutta : https://fr.wikipedia.org/wiki/M%C3%A9thodes_de_Runge-Kutta

[2] Méthodes de Runge-Kutta : https://femto-physique.fr/analyse-numerique/runge-kutta.php

[3] Méthodes numériques pour la physique : https://www.physagreg.fr/methodes-numeriques-euler-runge-kutta.php